Week 3 — Wednesday Lab: Navigation Bar & Role Protection

IM102 – Advanced Database Systems | 3 Hours | Computer Lab

Today's Goal

Build a navbar that shows different links for admins vs staff. Lock admin pages so staff can't access them.

Example: Role-Based Navbar

Instead of one static navbar, use your auth functions to show different content:

```
<nav>
    <a href="index.php">Home</a>
    <a href="report.php">Reports</a>

    <?php if (isAdmin()): ?>
        <a href="add.php">+ Add</a>
        <a href="users.php">Users</a>
    <?php endif; ?>

    <span style="margin-left:auto;">
        <?= getUsername() ?> (<?= $_SESSION['role'] ?>)
        <a href="logout.php">Logout</a>
    </span>
</nav>
```

Staff see Home and Reports. Admins see everything. The PHP `if` controls what HTML gets sent to the browser — staff never even receive the admin links.

Example: Page Protection

At the top of admin-only pages:

```
<?php
require_once 'auth.php';
requireAdmin();  // dies with "Access denied" if not admin
?>
```

That one function call replaces manually checking `$_SESSION['role']` on every page.

Your Turn

Create `navbar.php` with a dark background, role-based links, username + role badge on the right, and logout. Include it in every page's `<body>`.

Add `requireAdmin()` to `add.php`, `edit.php`, and `delete.php`. Keep `requireLogin()` on `index.php` and `report.php` — staff can view these.

Test as both roles. As staff, try directly visiting `add.php`, `edit.php?id=1`, `delete.php?id=1` — you should get "Access denied" every time. If you can access them, your protection isn't working.